ErisTutor
Hệ con Memory (Bộ nhớ nhận thức)

Báo cáo tri thức trích xuất từ mã nguồn

Mã tài liệu:
ERISTUTOR_20261107_V02_M01_MEMORY_SUBSYSTEM

Phiên bản:
V02

Mốc đối chiếu:
07/11/2026 - 15:00 (UTC+07:00)

Nguồn:
eris-ai-20261107_130547_937.zip -> eris/src/memory/

Quy mô:
105 module Python - 29,504 dòng mã

Phạm vi:
Trích xuất chỉ đọc; không sửa mã nguồn.

1. Memory là gì trong ErisTutor

Memory (eris/src/memory/) là hệ con bộ nhớ nhận thức của Eris AI - nền tảng mà ErisTutor dựa lên. Đây không phải log hội thoại đơn thuần, mà là một kiến trúc nhiều tầng: tiếp nhận quan sát, chưng cất thành claim/fact có nguồn gốc (provenance), lưu vào nhiều loại kho ký ức, rồi truy hồi lại theo truy vấn có kiểm soát để đưa vào nhận thức/câu thoại.

Toàn hệ gồm 105 module (~29,504 dòng), là một trong ba cụm nặng nhất của codebase cùng với runtime và cortex. Theo kế hoạch rebuild theo dependency, memory thuộc tầng "Core Cognition" và nằm trong vòng phụ thuộc chéo llm - memory - cortex - runtime, nên được giao trọn cho một chủ sở hữu thay vì chia nhỏ.

Vai trò với phần học tập (ErisTutor)

- living_user/ giữ sở thích, mục tiêu, cam kết do người dùng nêu rõ - đưa vào LearnerProfile, KHÔNG thay thế bằng chứng làm bài.
- Ranh giới ADR: memory chỉ cấp "đã quan tâm / đã nêu", còn mastery phải đến từ LearningService deterministic. Không trộn hai loại.
- Portrait do Gemini viết là read-model có provenance, không phải sự thật gốc.

2. Nguyên tắc thiết kế xuyên suốt

Provenance là bắt buộc
Mỗi claim mang nguồn gốc từ ingest -> commit -> reload. Metadata do runtime ingress cấp; chuỗi do model sinh KHÔNG được thay origin hay quyền xác nhận.

Một nguồn sự thật
Không tạo memory store thứ hai. subject/repository.py là SQLite source of truth cho Subject Core; các kho còn lại là JSON atomic.

Ký ức nên "nhàm chán"
Canonical long-term cố tình đơn giản: JSON atomic, evidence ID, confidence, status, liên kết mâu thuẫn và decay. Bản "thú vị" thường là bản tự tin nhớ sai.

Dreaming có kiềm chế
Hợp nhất kiểu dream là củng cố mẫu có độ trễ, không phải callback sau mỗi lượt; lượt giá trị thấp không phát sinh yêu cầu long-term.

Đọc không suy diễn
Các đường đọc (episode projection) là view nhỏ, deterministic; không chạy inference hay embedding trên read path.

Cảm xúc là pressure, không phải fact
Affective residue lưu dạng áp lực (tension, attachment, recoil) ảnh hưởng nhịp nói/im lặng, không tự động thành câu thoại công khai.

3. Cấu trúc theo nhóm

Nhóm (thư mục)	Số file	Dòng mã
(root) - lõi memory	69	19,443
living_user/	24	7,743
subject/	3	877
understanding/	5	1,052
claims/	2	311
judgement/	2	78
Tổng	**105**	**29,504**

4. Lõi memory (root) theo chức năng

Nền tảng & bản ghi

__init__.py (52 dòng)
Public memory API for ERIS. Use `MemoryKernel` from the package root for new integration. Legacy core/query/commit types are intentionally no longer re-exported here. Import them from `memory.memory_cognition` explicitly when compatibility

memory_record.py (414 dòng)
Lớp: MemoryRecord

memory_models.py (222 dòng)
Lớp: MemoryTarget, SurfaceMode, DecayPolicy, MemoryCommitKind, MemoryKind, MemoryCommitDecision, RecallRequest, RecallAnswer

schemas.py (490 dòng)
Lớp: MemoryTarget, DecayPolicy, PrivacyLevel, MemoryStatus, CommitOutcome, CommitDecision, MemoryRecordBase, EventLedgerEntry (+9)

json_store_io.py (106 dòng)
I/O JSON dùng chung cho các memory store. Mục tiêu: - một lock theo đường dẫn để nhiều thread trong cùng process không ghi đè nhau; - temp file luôn duy nhất; - flush + fsync trước os.replace; - retry ngắn khi Windows/antivirus tạm giữ file

persistence.py (340 dòng)
Lớp: MemoryPersistenceConfig, MemoryPersistenceError, JsonMemoryPersistence

name_validation.py (117 dòng)

text_normalize_vi.py (65 dòng)

semantic_text.py (197 dòng)
Lớp: TextSemanticFrame

memory_services.py (97 dòng)
Lớp: MemoryRuntimeServices

Kho ký ức lõi

episodic_memory.py (666 dòng)
Episodic Memory: chuyện cụ thể đã xảy ra. Episodic không phải semantic fact. Nó giữ cảnh, lượt, outcome, evidence và độ nổi bật. Một câu user sửa sai hôm nay là episode. Một fact ổn định chỉ lên semantic sau khi commit phù hợp. Bản này sửa
Lớp: RecallScore, EpisodicMemoryItem, EpisodicMemoryStore

semantic_memory.py (1903 dòng)
Semantic Memory: bộ nhớ fact đã được chưng cất cho ERIS. File này cố tình KHÔNG làm event log. Semantic memory chỉ nên giữ những thứ đã đủ ổn định để ERIS có thể nhớ gọn: - fact về user, ERIS, project, thế giới, kỹ thuật - sở thích / thói q
Lớp: SemanticRecallCue, SemanticRecallHit, FactProfile, SemanticMemoryItem, SemanticMemoryStore

procedural_memory.py (195 dòng)
Procedural Memory: cách xử lý đã học. Đây là nơi lưu rule kiểu: khi user sửa hiểu lầm, trả lời trực tiếp trước; khi evidence yếu, không claim nhìn thấy màn hình. Procedural là policy học được, không phải fact về user.
Lớp: ProceduralMemoryItem, ProceduralMemoryStore

relational_memory.py (196 dòng)
Relational Memory: nhịp quan hệ, rupture, repair, khoảng cách. Đây không phải nơi lưu "user is sad". Nó lưu pattern có bằng chứng: user sửa ERIS khi overread, user thích câu trả lời kỹ thuật trực tiếp, rupture cần repair. Tức là ký ức quan
Lớp: RelationalMemoryItem, RelationalMemoryStore

repair_memory.py (242 dòng)

Repair Memory: ghi lỗi, sửa sai, suppression rule tương lai. Nếu user nói "không phải", hệ thống không được chỉ xin lỗi rồi quên. Nó phải lưu loại lỗi, assumption sai, evidence, chiến lược sửa. Đây là nơi ERIS học bớt ngu theo thời gian, mộ

Lớp: RepairMemoryItem, RepairMemoryStore

affective_residue_memory.py (207 dòng)

Affective Residue Memory: dư âm cảm xúc dưới dạng pressure. Không lưu "ERIS buồn" như fact công khai. Lưu pressure: tension, attachment residue, exposure recoil, unfinished loop. Dư âm ảnh hưởng nhịp nói/im lặng, không tự động thành câu tho

Lớp: AffectiveResidueItem, AffectiveResidueMemoryStore

unresolved_memory.py (325 dòng)

Lớp: UnresolvedMemoryCandidate, UnresolvedMemoryStore

active_state_snapshot.py (166 dòng)

Active State Snapshot cho ERIS. Snapshot giữ trạng thái hiện tại có thể overwrite: goal đang theo, session arc, affect/homeostasis hiện thời, relation state, project active, thread đang mở. Nó không phải event ledger và không phải semantic

Lớp: ActiveStateSnapshot, ActiveStateStore

event_ledger.py (235 dòng)

Append-only Event Ledger cho ERIS. Tầng này giữ raw truth: tín hiệu thô, evidence đã tạo ra, frame liên quan, và metadata audit. Nó không kết luận tâm lý, không merge ký ức, không sửa quá khứ. Nếu một module muốn lưu sự kiện, nó phải tạo St

Lớp: EventLedgerRecord, EventLedger

canonical_long_term.py (297 dòng)

Persistent canonical long-term stores for ERIS V3. The stores are intentionally boring: atomic JSON, evidence IDs, confidence, status, contradiction links and decay. Long-term memory should be boring. The exciting version is usually the one

Lớp: CanonicalMemoryRecord, CanonicalRecordStore, BeliefStore, WorldviewMemoryStore, SocioculturalMemoryStore, SpatialMemoryStore, TemporalRhythmMemoryStore, BodyRhythmMemoryStore (+3)

belief_store.py (3 dòng) [stub]

worldview_memory.py (3 dòng) [stub]

sociocultural_memory.py (3 dòng) [stub]

spatial_memory.py (3 dòng) [stub]

temporal_rhythm_memory.py (3 dòng) [stub]

body_rhythm_memory.py (3 dòng) [stub]

tom_history.py (3 dòng) [stub]

self_continuity_memory.py (3 dòng) [stub]

Claim & provenance

semantic_claim.py (291 dòng)

Lớp: SemanticClaim, ExtractedClaim

claim_extractor.py (75 dòng)

Lớp: ClaimExtractor

claim_entailment.py (130 dòng)

Lớp: ClaimEntailmentDecision, ClaimEntailmentEngine

claim_provenance.py (115 dòng)

Nguồn của một claim xuyên suốt ingest -> commit -> reload. Metadata do *runtime ingress* cấp; chuỗi do model viết KHÔNG được dùng để thay origin, roots hoặc quyền xác nhận. Không tạo memory store thứ hai.

provenance_policy.py (126 dòng)

Policy provenance và trạng thái nhận thức cho ký ức public.

Lớp: PublicMemoryMode, ProvenanceDecision, ContestedClaimStore, MemoryProvenancePolicy

Predicate & ontology

predicate_catalog.py (518 dòng)
Lớp: FrameRoute, OntologyPolicy, QueryLexicon, RetentionPolicy, SurfacePolicy, PredicateDefinition

predicate_ontology.py (187 dòng)
Lớp: EntailmentType, PredicateSpec

predicate_registry.py (850 dòng)
Lớp: PredicateConflictError, PredicateResolution, PredicateRegistry

Truy vấn ký ức

query_intent.py (230 dòng)
Lớp: SemanticQueryIntent, QueryTargetFrame, QueryIntentParserProtocol, SemanticQueryIntentCompiler

memory_query_ir.py (369 dòng)
Lớp: QueryIntent, RoleRef, ActRef, RelationRef, SurfaceMode, MemorySourceKind, MemorySelector, MemoryRelation (+4)

memory_query_compiler.py (364 dòng)
Lớp: CompileDebug, MemoryQueryCompileResult, MemoryQueryCompiler

memory_query_executor.py (489 dòng)
Lớp: MemoryQueryExecutor

memory_query_semantics.py (370 dòng)
Lớp: EmbeddingProvider, SemanticQueryHint, MemoryQuerySemanticHintEngine

memory_query_slot_parser.py (587 dòng)
Lớp: SlotSignal, SlotParseResult, MemoryQuerySlotParser

memory_query_llm_parser.py (215 dòng)
Lớp: MemoryQueryLLMClient, LLMParseDecision, MemoryQueryLLMParser

memory_query_repair.py (277 dòng)
Lớp: MemoryQueryRepairItem, RepairPriorResult, MemoryQueryRepairStore

memory_retrieval_engine.py (157 dòng)
Lớp: MemoryStores, MemoryRetrievalEngine

memory_recall_policy.py (264 dòng)
Lớp: RecallPolicyConfig, MemoryRecallSurfacePolicy

embedding_index.py (505 dòng)
memory/embedding_index.py ========== In-memory embedding index dùng chung cho semantic memory và query compiler. Điểm quan trọng của bản này: - Tách encode QUERY và DOCUMENT. Qwen3/E5 cần hai đường khác nhau. - Tự chọn poolin
Lớp: EmbeddingIndex

Chấm điểm & nạp ký ức

memory_admission.py (157 dòng)
Memory admission policy V41. Retrieval chỉ nói "record này có vẻ liên quan". Admission mới trả lời câu khó hơn: "record này có đủ tư cách bước vào cognition/prompt của lượt hiện tại hay không?" Tách hai việc này ra để audit/runtime/transien
Lớp: MemoryAdmissionDecision, MemoryAdmissionPolicy

memory_scoring.py (339 dòng)
Lớp: LayerProfile, MemoryScore, MemoryScorer

memory_activation.py (268 dòng)
Lớp: MemoryActivation, ScoredMemoryCandidate, MemoryRetrievalResult, WorkingMemoryPolicy, WorkingMemoryGate, MemoryActivationBuilder

memory_candidate.py (178 dòng)
Lớp: MemoryCandidate, MemoryLayerAdapter

memory_worthiness.py (135 dòng)
Lớp: WorthinessDecision, MemoryWorthinessPolicy

memory_cue.py (332 dòng)
Lớp: MemoryCue, MemoryCueBuilder

memory_packet_builder.py (174 dòng)
Lớp: MemoryPacket, MemoryPacketBuilder

memory_feed_trace.py (137 dòng)
Trace V41 cho những gì memory thực sự được feed vào cognition/prompt. File này cố tình KHÔNG lưu nguyên text memory. Trace chỉ giữ ID/layer/reason/score để debug contamination mà không tạo thêm một bản sao dữ liệu người dùng. Rác audit khôn
Lớp: MemoryFeedTraceJournal

memory_surface_filter.py (39 dòng)
Syntax-only exclusions for session-scoped social permission text. PDF annotation, pages 12-13 and 20: this module protects the persistence boundary. It does not classify open conversational meaning and never affects public wording.

Hợp nhất & bảo trì

memory_commit_policy.py (1063 dòng)
Lớp: MemoryCommitPolicy

memory_retention_policy.py (134 dòng)
Lớp: RetentionDecision, MemoryRetentionPolicy

memory_consolidator.py (81 dòng)
Lớp: ConsolidationResult, MemoryConsolidator

memory_maintenance.py (248 dòng)
Lớp: MemoryMaintenanceRunner

memory_pruner.py (123 dòng)
Lớp: PruneDecision, MemoryPruner

dream_scheduler.py (137 dòng)
Canonical reflection/dream scheduling and consolidation for ERIS V3. Dream is delayed pattern consolidation, not a decorative callback after every turn. The scheduler therefore emits no long-term requests for ordinary low-value turns. Human
Lớp: DreamScheduleDecision, DreamConsolidationEngine

dream_consolidation.py (3 dòng) [stub]

Lõi nhận thức & kernel

memory_cognition.py (738 dòng)
Memory Cognition Core cho ERIS. Bản v2: - Commit protocol vẫn giữ nguyên: module khác chỉ tạo StateCommitRequest. - Recall không còn tự đo resonance bằng token overlap trong MemoryCognitionCore. - MemoryCognitionCore giờ là facade: route co
Lớp: MemoryQuery, MemoryCommitDecision, CommitPolicy, MemoryCognitionCore

memory_kernel.py (843 dòng)
memory_kernel.py ================ MemoryKernel là cổng thần kinh duy nhất cho memory runtime. Mục tiêu của file này: - Runtime không gọi trực tiếp MemoryCommitPolicy, MemoryQueryExecutor, PacketBuilder, CandidateStore, Pruner lung tung nữa.
Lớp: MemoryKernelDebugSnapshot, MemoryKernel

memory_kernel_contracts.py (265 dòng)
Lớp: PublicMemoryContext, MemoryKernelConfig, MemoryIngestPlan, MemoryIngestResult, MemoryAnswerResult, MemoryMaintenanceReport

memory_ai_curator.py (532 dòng)
memory/memory_ai_curator.py ========================== AI curator cho memory — gọi Gemini để làm 2 việc rule không làm được: 1. Episodic distillation: Nhiều episode tương đồng → 1 semantic fact ổn định. Ví dụ: 5 lần user đề cập "thích câu
Lớp: CuratorResult, MemoryAICurator

memory_turn_annotator.py (407 dòng)

Lớp: TurnAnnotation, TurnAnnotator

memory_turn_graph.py (435 dòng)

Lớp: TurnRole, TurnSpeechAct, TurnEdgeType, FactMention, TurnNode, TurnEdge, TurnGraph

5. Các gói con chuyên biệt

living_user/ - Chân dung người dùng sống

__init__.py (89 dòng)
Living user memory public API.

action_memory.py (582 dòng)
Goal, commitment và workflow memory cho ERIS. Profile trả lời câu hỏi "người dùng là ai / thích gì". Module này giữ phần còn lại mà một trợ lý thật sự cần: người dùng đang cố làm gì, ai đã hứa làm gì và cách làm việc nào nên được tái sử dụn
Lớp: ActionMemoryOperation, GoalRecord, CommitmentRecord, WorkflowRecord, ActionContext, ActionMemoryStore

authority.py (301 dòng)
Nguồn thẩm quyền cho các thao tác người dùng lên memory. Living-user profile, semantic index và panel chỉ là projection. Khi người dùng sửa, xác nhận, đổi phạm vi hoặc yêu cầu quên, thao tác đó được ghi append-only vào ledger này trước. Nhờ
Lớp: UserMemoryAuthorityEvent, AuthorityDecision, UserMemoryAuthorityLedger

benchmark.py (93 dòng)
Benchmark nhỏ, reproducible cho living user memory. Unit test kiểm tra từng hàm. Benchmark kiểm tra hành vi end-to-end: setup profile, compile context, đo recall của path cần có và false-memory từ path bị cấm. File case là JSONL để người dù
Lớp: BenchmarkMetrics, LivingUserMemoryBenchmark

clarification.py (69 dòng)
Policy bất định cho memory dùng trong câu trả lời. Embedding chỉ tìm được ứng viên; policy này quyết định memory có được dùng thẳng, phải nói dè dặt, cần xác nhận hay nên bỏ qua. Mục tiêu là giảm false-memory, thứ nguy hiểm hơn nhiều so với
Lớp: MemoryUseDecision, MemoryClarificationPolicy

context_compiler.py (439 dòng)
Biên dịch user profile thành context packet theo đúng query hiện tại.
Lớp: ProfileContextCompilerConfig, ProfileContextCompiler

curator.py (629 dòng)
Bộ kiểm toán và dọn dẹp Living User Memory. Phân vai V68: - Extractor/dreamer chỉ tạo evidence-backed working records. - Embedding chỉ gom hàng xóm gần nghĩa để Gemini Web nhìn được cụm. - Gemini Web curator là authority duy nhất của *cấu t
Lớp: LivingUserCuratorConfig, LivingUserCuratorResult, LivingUserMemoryCurator

dreamer.py (258 dòng)
Dreaming/consolidation cho living user profile. Dreamer không chạy sau mọi turn. Nó đọc episode, semantic facts và profile hiện có, rồi đề xuất một tập patch nhỏ. Patch vẫn phải qua validator/store.
Lớp: UserProfileDreamConfig, UserProfileDreamer

episode_builder.py (198 dòng)
Xây episode từ các lượt hội thoại hoặc event ledger.
Lớp: EpisodeBuilderConfig, EpisodeModelError, UserEpisodeBuilder

episode_projection.py (86 dòng)
Small, deterministic episode views. No inference or embedding on read paths.

episode_rollup.py (222 dòng)
Durable, scoped rolling recaps. Inference runs only from maintenance. The recap and its source coverage are one atomic document. Sources remain in the episode store, including its cold archive; recap coverage is not permission to skip profi
Lớp: EpisodeRollupConfig, EpisodeRollupStore

episode_store.py (323 dòng)
Episode store cho living user memory.
Lớp: UserEpisodeStore

evidence_context.py (84 dòng)

Bounded, role-labelled source context for memory interpretation, not a selector of meaning.

extractor.py (234 dòng)

Trích xuất patch profile từ semantic claim và tín hiệu hội thoại. Extractor có hai nhánh: - Nhánh deterministic cho dữ kiện đã được memory semantic parser xác nhận. - Nhánh model adapter cho diễn giải hội thoại; không tự học bằng cue từ khó

Lớp: UserProfileModelAdapter, ProfileExtractionConfig, UserProfileExtractor

gemini_adapter.py (122 dòng)

Living User/GC dùng lane Gemini Web của service đang giữ phiên đăng nhập. Cookie và gemini_webapi thuộc service Gemini, không thuộc Python portable. Không fallback sang DeepSeek khi service lỗi: caller phải giữ evidence và retry.

Lớp: GeminiWebUserProfileAdapter

legacy_projection.py (131 dòng)

Projection tương thích API ``cortex.private_mind.UserModelStore``. Mục tiêu là cho private mind đọc living profile mới mà không cần thay cả subsystem trong một lần. Projection không tạo nguồn chân lý thứ hai; mọi mutation đi thẳng về UserPr

Lớp: LivingUserModelProjection

models.py (574 dòng)

Kiểu dữ liệu canonical cho living user memory. Tầng này không gọi model và không chạm persistence. Nó chỉ mô tả dữ liệu mà các tầng khác được phép trao đổi. Giữ contract gọn giúp tránh chuyện mỗi module phát minh một kiểu ``dict`` khác nhau

Lớp: UserMemoryScope, EvidenceReference, ProfileRecord, ProfileOperation, UserEpisode, ProfileContextItem, ProfileContextPacket, DreamReport

portrait_store.py (204 dòng)

Chân dung người dùng do Gemini viết, lưu bền vững nhưng KHÔNG phải sự thật gốc. - Fact/episode từ Living User Memory là nguồn duy nhất được phép dùng. - Mở panel chỉ đọc đĩa: không chờ mạng, không gọi Gemini trên luồng UI. - Khi nguồn thay

Lớp: SavedGeminiMemoryPortrait

presenter.py (499 dòng)

Character-facing projection cho Living User Memory. File này KHÔNG tạo fact mới và KHÔNG đọc raw chain-of-thought. Nó chỉ nhận: - canonical memories đã được lọc cho UI; - episode đã được Living User Memory tổng hợp; - một ``expression_lens`

Lớp: MemoryPortraitItem, MemoryPortraitSection, MemoryPortrait, MemoryPortraitPresenter

runtime_adapter.py (204 dòng)

Adapter LLM service -> structured living user memory operations. Không bắt buộc phải bật. Khi model không sẵn sàng, service vẫn chạy bằng semantic claims và deterministic consolidation. Remote inference chỉ làm giàu profile, không trở thành

Lớp: CuratorInferenceError, LlmServiceUserProfileAdapter

service.py (1331 dòng)

Facade cấp cao cho living user memory. Runtime chỉ nên gọi service này, thay vì tự mở profile store, episode store rồi trộn business rule vào chat cycle. Một facade có ích là facade làm giảm quyền của caller, không phải chỉ đổi tên sáu meth

Lớp: LivingUserMemoryConfig, UserTurnObservation, LivingUserMemoryService

sources.py (121 dòng)

Theo dõi và giải thích nguồn của living user memory.

Lớp: MemoryExplanation, MemorySourceTracker

store.py (782 dòng)

Persistence và mutation engine cho living user profile. Store chỉ nhận ``ProfileOperation`` đã được validator thông qua. Nó không gọi LLM, không đoán tâm lý và không tự tạo nội dung. Nhờ vậy mọi thay đổi đều audit được, undo được và có thể

Lớp: UserProfileStoreConfig, ApplyResult, UserProfileStore

trace.py (168 dòng)

Trace journal và replay debugger cho living user memory. Journal giải thích được tại sao một profile item xuất hiện, context nào đã được inject và operation nào bị từ chối. Sensitive value được redacted trước khi ghi; trace phục vụ debug, k

Lớp: MemoryTraceEvent, MemoryTraceJournal, ReplayReport, MemoryReplayDebugger

subject/ - Subject Core (SQLite source of truth)

__init__.py (17 dòng)
Subject Core persistence.

repository.py (812 dòng)
SQLite source of truth for Subject Core events and projections. V6 provides migration/checksum inspection, fenced cross-process idempotency claims, consistent backup and a machine-readable integrity report. Databases, much like humans, bene
Lớp: StaleRevisionError, EventIntegrityError, IdempotencyClaim, StoredEvent, SubjectRepository, AsyncPersistenceActor

unit_of_work.py (48 dòng)
SQLite unit of work with explicit transaction ownership.
Lớp: UnitOfWork

understanding/ - Tầng đọc nghĩa trung gian

__init__.py (16 dòng)
Tầng đọc nghĩa trung gian cho memory intake.

entity_linker.py (109 dòng)

meaning_frame.py (192 dòng)
Lớp: EntityRef, MeaningFrame, MeaningParseResult

polarity_temporal.py (109 dòng)
Lớp: SurfaceFeatures

semantic_parser.py (626 dòng)
Lớp: SemanticParser, ModelSemanticParser, StructuredFrameParser, FallbackSurfaceSemanticParser

claims/ - Tầng logic claim

__init__.py (15 dòng)
Tầng claim logic: build claim, ontology, entailment.

claim_builder.py (296 dòng)
Lớp: ClaimBuilder

judgement/ - Tầng phán xét ký ức

__init__.py (17 dòng)
Tầng phán xét memory: worthiness, retention, consolidation.

consolidation.py (61 dòng)
Lớp: ConsolidationDecision